

Lab 1 — Requirements Engineering & UML Use-Case Modelling

Problem Statement #05: Academic Elective Bidding & Allocation System

Name: Roopesh

SRN: PES1UG24AM391

1. Requirements Table

Functional Requirements

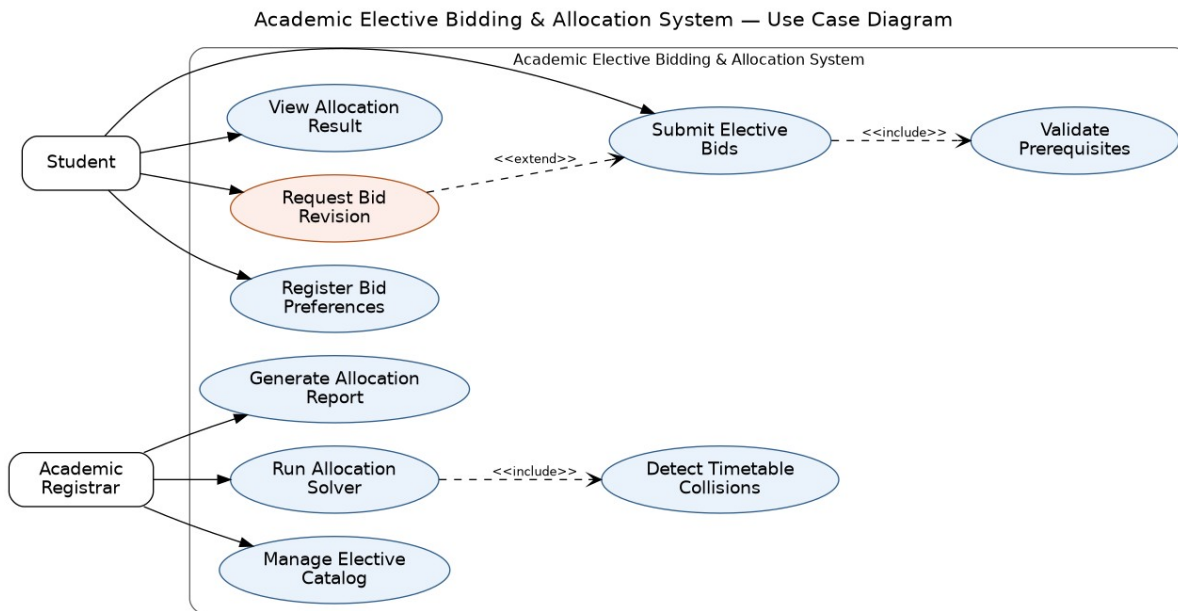
ID	Priority	Description	Acceptance Criteria	Rationale
FR-001	High	The system shall allow a student to distribute exactly 100 bidding credits across their ranked elective preferences and shall validate that all prerequisite courses for each chosen elective have been completed.	Pass: Credits deduct accurately across preferences and prerequisites are validated. Fail: Total allocated credits exceed 100, or an elective with an unmet prerequisite is accepted into the bid.	This is the core input mechanism the whole allocation process depends on, so it has to be airtight before any solving happens.
FR-002	High	The system shall run an allocation solver that assigns each student to electives based on their bid credits and ranked preferences, while ensuring no student is assigned two electives with overlapping class timings.	Pass: Every student's final allocation is conflict-free and reflects their highest-priority affordable preference. Fail: A student receives two electives with a timetable clash, or an allocation ignores bid ranking without reason.	Directly delivers on the promise of the system — automated, fair allocation without manual timetable checking.
FR-003	Medium	The system shall allow a student to submit a bid revision request before the bidding deadline, replacing their previously submitted preference list and credit distribution.	Pass: The old bid is fully overwritten by the new one and credit totals are re-validated. Fail: Both the old and new bids are counted, or a revision is accepted after the deadline has passed.	Students realistically change their minds once they see seat availability or talk to seniors, so a locked-in bid on day one is unrealistic.
FR-004	Medium	The system shall allow the Academic Registrar to add, edit, or remove electives in the catalog, including seat capacity, timetable slot, and prerequisite course codes.	Pass: Catalog changes are reflected immediately in what students can bid on. Fail: A removed or edited elective still appears to students in its old form.	The registrar needs to react to last-minute changes (a professor drops out, a lab section fills up) without needing a developer involved.
FR-005	Medium	The system shall generate an allocation summary report for the Academic Registrar showing, per elective, the number of seats filled, number of students waitlisted, and average bid credits spent.	Pass: Report totals reconcile with the actual allocation output. Fail: Reported seat counts do not match the number of students actually allocated to that elective.	The registrar needs this to spot under-subscribed electives and adjust capacity or offerings for the next cycle.

Non-Functional Requirements

ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Performance & Security	The final elective allocation solver shall process 5,000 student bids and resolve all schedule conflicts in under 30 seconds, and all bid data in transit and at rest shall be encrypted.	High	Pass: Benchmarking confirms the solver completes within 30 seconds for 5,000 simulated bids, and security scans confirm encryption is applied. Fail: Solve time exceeds 30 seconds, or bid data is found unencrypted.	Allocation typically runs once for the whole department at a fixed time, so students are all waiting on this one run — it can't be slow, and bid credits are sensitive enough to warrant encryption.
NFR-002	Reliability & Auditability	Once the bidding deadline passes, a student's submitted bid must	High	Pass: No bid record can be modified post-deadline	Allocation disputes are inevitable, and the

ID	Type	Description	Priority	Acceptance Criteria	Rationale
		become immutable, and any change to bid or allocation data after that point must be recorded in an audit log with a timestamp and actor ID.		outside of an audited registrar override, and every override is logged. Fail: A bid is silently altered after the deadline with no log entry.	registrar needs a tamper-evident trail to resolve 'why did I get this elective' complaints fairly.

2. UML Use-Case Diagram



Use-case diagram for the Academic Elective Bidding & Allocation System, showing the Student and Academic Registrar actors, all primary use cases, an <<include>> relationship (Submit Elective Bids → Validate Prerequisites, Run Allocation Solver → Detect Timetable Collisions) and an <<extend>> relationship (Request Bid Revision → Submit Elective Bids).

3. Use-Case Flow Specification

Use Case: Submit Elective Bids (includes Validate Prerequisites)

Actors

Primary Actor: Student. Secondary Actor: Academic Registrar (owns the elective catalog the bid is validated against).

Preconditions

- The student is logged in and the bidding window for the current semester is open.
- The elective catalog for the semester has been published by the Academic Registrar, including seat capacity, timetable slots, and prerequisites for each elective.
- The student has not already submitted a final, locked bid for this cycle (or is submitting a revision before the deadline — see Request Bid Revision).

Postconditions

- Success: The student's ranked elective list and credit distribution (summing to 100) are saved; each listed elective has passed the prerequisite check; the bid is queued for the allocation solver.
- Failure: No bid record is saved, and the student's previous bid (if any) remains unchanged.

Main Success Scenario

1. Student opens the bidding page and views the published elective catalog.
2. Student ranks their preferred electives and distributes 100 bidding credits across them.
3. Student submits the bid.

4. System includes the Validate Prerequisites use case to check that the student has completed the prerequisite course(s) for every elective on the list.
5. System confirms the total allocated credits equal 100 and do not exceed the limit.
6. System saves the bid record and displays a confirmation with a submission timestamp to the student.
7. Use case ends in success.

Alternate Flow: Prerequisite Not Met

Trigger: At step 4, the system finds that the student has not completed the prerequisite for one of the electives on their list.

1. System flags the specific elective(s) that failed the prerequisite check and rejects the bid as submitted.
2. System displays which elective(s) are ineligible and why, and returns the student to their ranked list for editing.
3. Student either removes the ineligible elective(s) and redistributes credits among the remaining choices, or cancels the submission.
4. If the student edits and resubmits, the use case re-enters the Main Success Scenario at step 4. If the student cancels, the use case ends in failure with no bid saved.